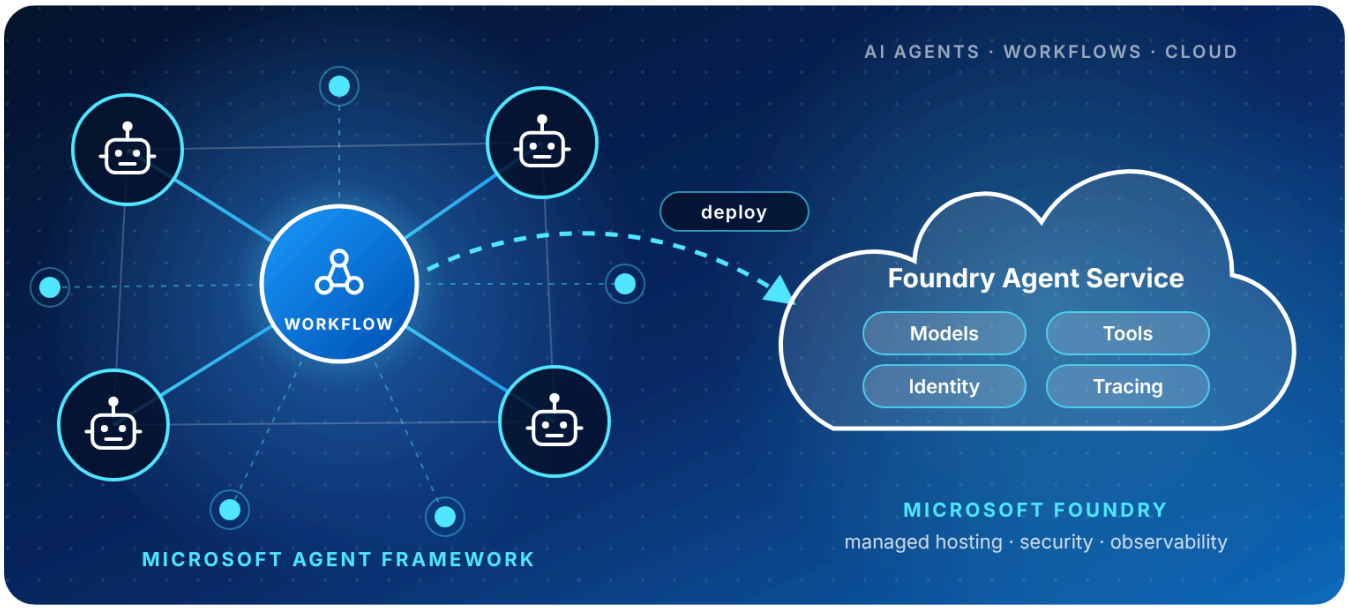
Microsoft Agent Framework (MAF) and Microsoft Foundry Agent Service

Nov 10, 2026 · @Mradul

Understanding Microsoft's Approach to Building, Orchestrating, and Deploying AI Agents

Technical Research and Learning Report



Prepared by	Organization	Date
[Author Name]	[Organization]	[Date]

1. Introduction to AI Agents

An AI agent is software that uses a large language model (LLM) to pursue a goal: it decides what to do, calls tools to do it, and reports the result. A chatbot replies to one message; an agent keeps working across steps until the task is done.

Aspect	Traditional chatbot	AI agent
Behavior	Replies to a message	Works toward a goal over several steps
Knowledge	Scripts or training data	Live data fetched through tools
Actions	Few or none	Calls functions, APIs and other agents
Control flow	Fixed by the developer	Chosen by the model, within set limits

An agent has five working parts. The **LLM** reasons and decides. **Tools** (functions, APIs, search, MCP servers) let it act. **Memory** holds conversation history and stored facts. **Planning** breaks a goal into steps. **Orchestration** is the runtime loop that connects them and, in larger systems, coordinates several agents.

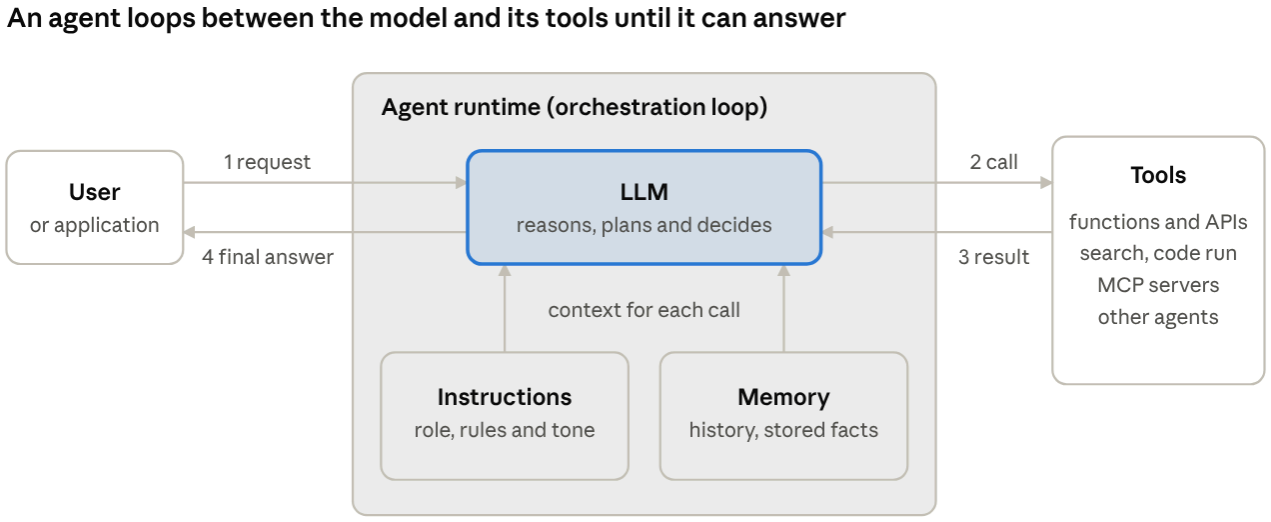


Figure 1 · how an AI agent works: model, context and tools in one loop

Steps 2 and 3 repeat as often as the model needs. Microsoft adds a caution: "If you can write a function to handle the task, do that instead of using an AI agent" (Agent Framework overview).

2. Introduction to Microsoft Agent Framework (MAF)

Microsoft Agent Framework is an open-source SDK for building AI agents and multi-agent workflows in .NET, Python and Go. It is a code library that runs wherever your application runs; it is not a cloud service.



Microsoft announced it on October 1, 2025 to solve three problems: fragmented open-source frameworks with incompatible APIs, code that works locally but not in the cloud, and missing enterprise features such as observability and security.

It is the direct successor to Semantic Kernel and AutoGen, built by the same teams. Microsoft Learn says it "combines AutoGen's simple agent abstractions with Semantic Kernel's enterprise features — session-based state management, type safety, middleware, telemetry — and adds graph-based workflows for explicit multi-agent orchestration." It builds on both projects and does not replace them: both remain supported, with migration guides from Semantic Kernel and from AutoGen.

Area	What it provides
Agents	LLM-driven agents that call tools and MCP servers
Harness Agent	A batteries-included agent for long, multi-step tasks
Workflows	Functional and graph-based execution paths that connect agents and functions
Integrations	Model providers, tools, memory, middleware, evaluation and UI frameworks

Design philosophy. Open standards (MCP, A2A, OpenAPI), research patterns made usable in production, open-source extensibility and enterprise readiness.

Status. Version 1.0 for .NET and Python shipped in April 2026 (Visual Studio Magazine). The Go SDK is in public preview.

3. Architecture of Microsoft Agent Framework

An agent wraps a model client with instructions, tools, middleware, context providers and session state; workflows sit above agents to coordinate several of them. Every agent exposes the same run interface.

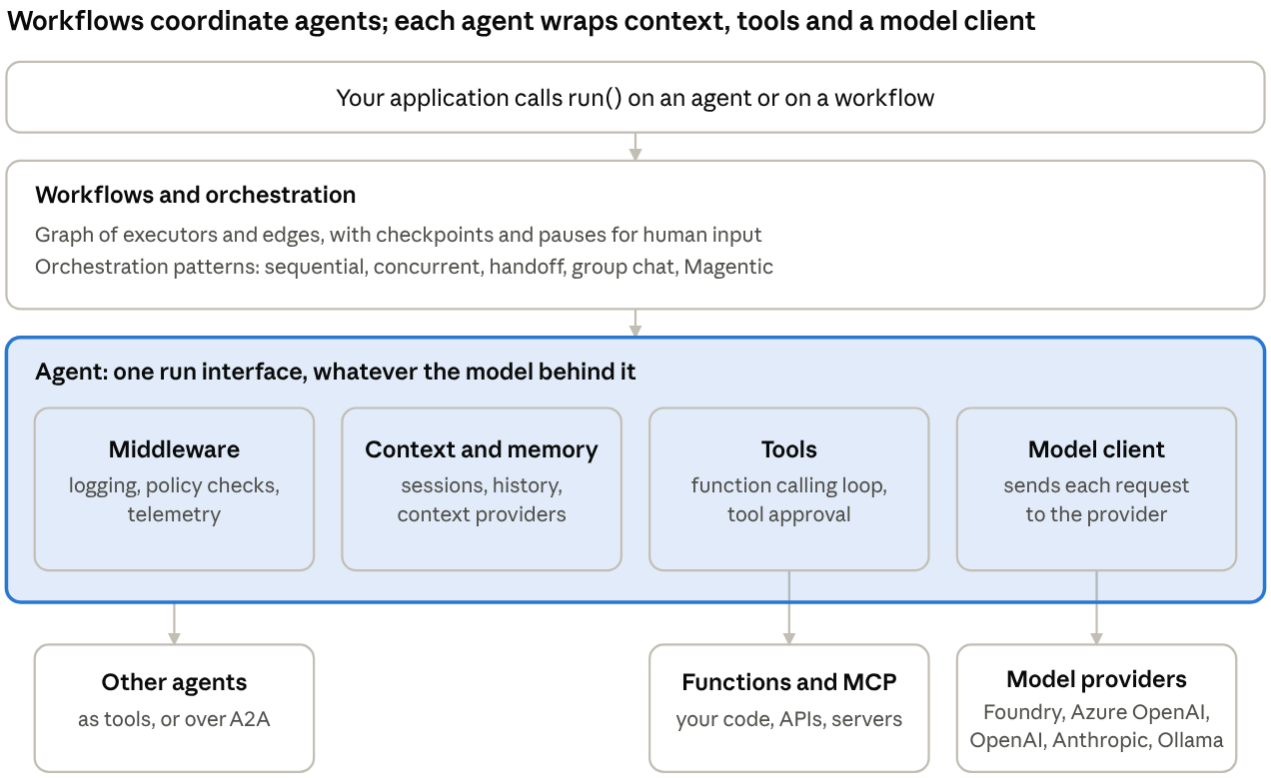


Figure 2 · Microsoft Agent Framework architecture

Component	Role	Key names
Agents	Reason and act	`Agent` (Python), `AIAgent` (.NET)
Model integration	Connects to an inference service; swappable	`FoundryChatClient`, `OpenAIChatClient`
Tools and function calling	Capabilities the model can request	Function tools, MCP tools, hosted tools
Memory and context	Conversation state and injected context	`AgentSession`, context providers
Workflows	Typed graph of executors and edges, with checkpoints	`WorkflowBuilder`
Agent orchestration	Prebuilt multi-agent patterns	Sequential, concurrent, handoff, group chat, Magentic
Multi-agent communication	Agents as tools, workflow edges, remote agents over A2A	`as_tool()`, `A2AAgent`

Use an agent for open-ended or conversational tasks, and a workflow when the steps are defined and their order must be controlled. Sources: Agents, Workflows, Tools.

4. Key Features of Microsoft Agent Framework

The framework scales from a few-line agent to checkpointed multi-agent workflows with human approval. This sample from the overview creates and runs an agent:

```
from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient
from azure.identity import AzureCliCredential

agent = Agent(
    client=FoundryChatClient(
        project_endpoint="https://your-foundry-
service.services.ai.azure.com/api/projects/your-foundry-project",
        model="gpt-5.4-mini",
        credential=AzureCliCredential(),
    ),
    name="HelloAgent",
    instructions="You are a friendly assistant. Keep your answers brief.",
)
result = await agent.run("What is the largest city in France?")
```

Feature	What it gives you	Example
Single-agent development	An agent from a model client and instructions	The sample above; `RunAsync` in .NET
Multi-agent collaboration	Five orchestration patterns; agents as tools	A triage agent hands off to a billing specialist
Workflow orchestration	Typed executors, conditional routing, fan-out, checkpoints	Extract, validate, then route failed documents to review
Tool integration	Function tools, MCP servers, hosted tools, Foundry Toolbox	`weather_agent.as_tool()` exposes one agent to another
Human-in-the-loop	Tools can require approval; workflows pause for input	`@tool(approval_mode="always_require")` on a refund tool
Streaming and async	All runs are asynchronous; updates stream as produced	`agent.run(..., stream=True)`; `RunStreamingAsync` in .NET

Feature	What it gives you	Example
Observability and debugging	OpenTelemetry traces for runs, model calls and tools	`configure_otel_providers()`; DevUI (preview) for local debugging

5. How Microsoft Agent Framework Works

One call to `agent.run()` (Python) or `RunAsync` (.NET) starts a loop between the model and your tools that ends when the model produces a final answer.

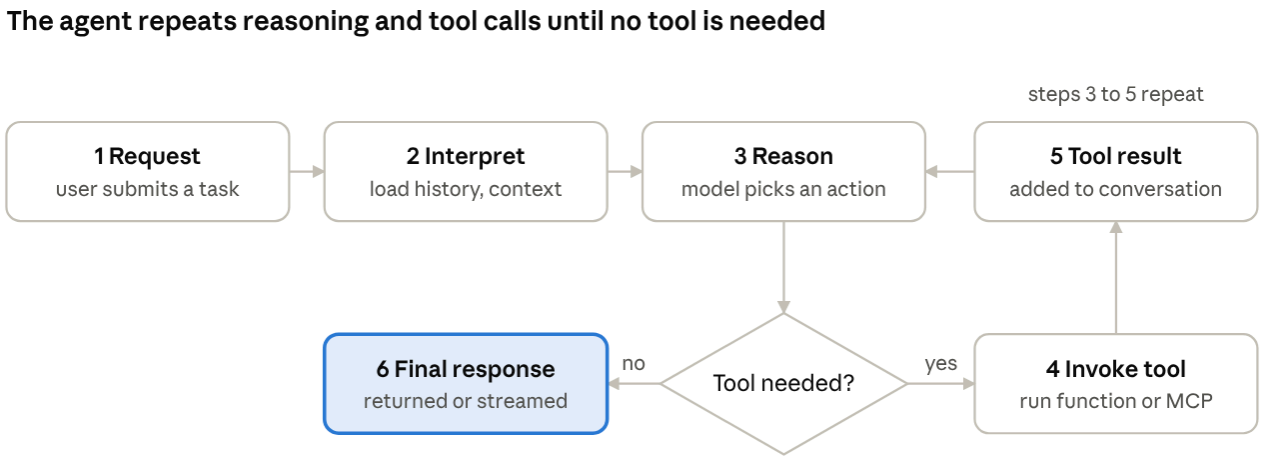


Figure 3 · Agent Framework execution lifecycle: six steps, one decision, one loop

1. **The user submits a request.** The application passes the message to the agent, optionally with a session.
2. **The agent receives and interprets it.** Middleware runs, and context providers load history and add context.
3. **The agent reasons and determines the next action.** The model returns either text or a function call.
4. **The agent invokes tools when required.** The framework runs the tool, pausing first if it needs approval.
5. **The agent processes tool results.** The result goes back to the model; steps 3 to 5 repeat as needed.
6. **The agent generates the final response.** The answer returns through middleware and the new messages are stored.

Example: asked "What is the weather like in Amsterdam?", the model calls a `get_weather` function tool, receives the result and writes the reply. In a workflow the same loop runs inside each executor, while the workflow engine decides which executor runs next (agent pipeline).

6. Practical Applications of MAF

Agent Framework suits any task where a model must combine reasoning with real systems, and its orchestration patterns map onto common business processes.

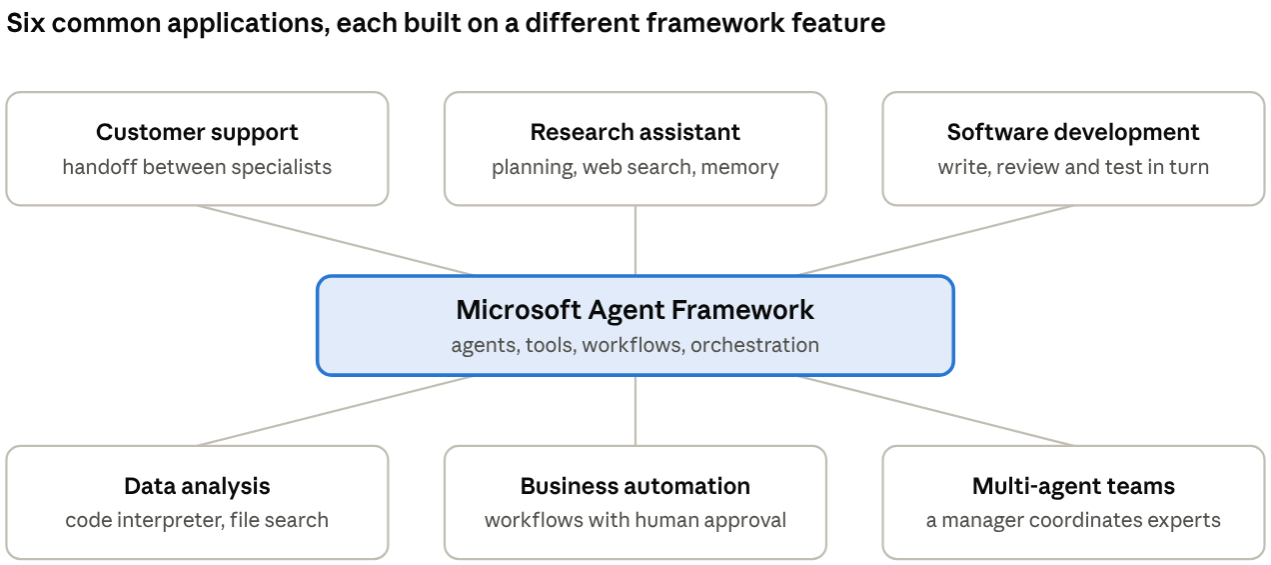


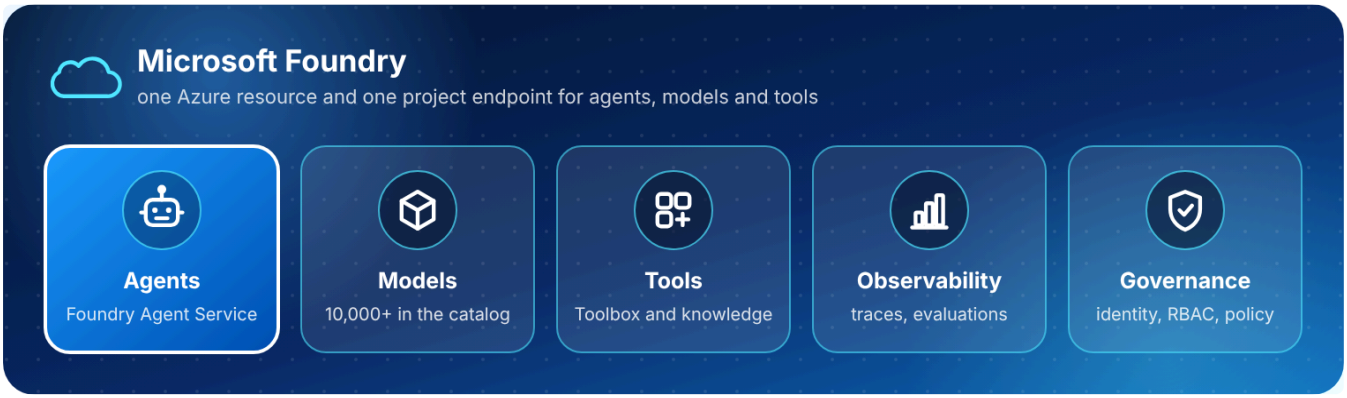
Figure 4 · use cases of Microsoft Agent Framework and the feature each relies on

Application	Example	Features used
Customer support agents	A triage agent passes a billing dispute to a specialist, which issues a refund after approval	Handoff, function tools, tool approval
Research assistants	Plans searches, gathers sources and writes a cited summary	Harness Agent, web search
Software development agents	One agent writes code, a second reviews it, a third runs tests	Sequential or group chat, MCP
Data analysis agents	Loads a dataset, runs calculations and explains the result	Code interpreter, file search
Business automation	Insurance claims pass fixed checks; large claims wait for an adjuster	Graph workflows, checkpoints
Multi-agent collaboration	A manager agent splits a goal among specialists	Magentic, concurrent

Early adopters named in Microsoft's announcement include KPMG ("Clara AI" for audit work), Commerzbank (customer support), BMW (vehicle telemetry analysis) and Fujitsu.

7. Introduction to Microsoft Foundry Agent Service

Foundry Agent Service is a managed cloud platform for building, deploying and scaling AI agents, with "any framework, any supported model from the Foundry model catalog, and a single entry point for model inference and tools" (overview).



It is the agents capability of Microsoft Foundry, the platform formerly named Azure AI Studio and Azure AI Foundry, which manages agents, models and tools as one Azure resource.

Agent type	What it is	Status
Prompt agent	Instructions, a model and tools saved as configuration. Foundry runs it; no code to maintain.	Available
Voice-based prompt agent	A real-time voice agent whose orchestration Foundry manages	Preview
Hosted agent	Your own code (Agent Framework, LangGraph, OpenAI Agents SDK or custom) in a Foundry-managed container	Available

Why a managed service. A production agent needs compute that scales, a secure endpoint, an identity, state storage, monitoring and safety controls. Agent Service supplies these so teams do not rebuild them for every agent.

Developing versus hosting. Developing decides what the agent does: instructions, tools and orchestration. Hosting decides where it runs, who may call it and how it is watched. Agent Framework addresses the first; Agent Service addresses the second.

8. Architecture of Microsoft Foundry Agent Service

Every capability is reached through one Foundry project endpoint, with the Agent Runtime at the centre.

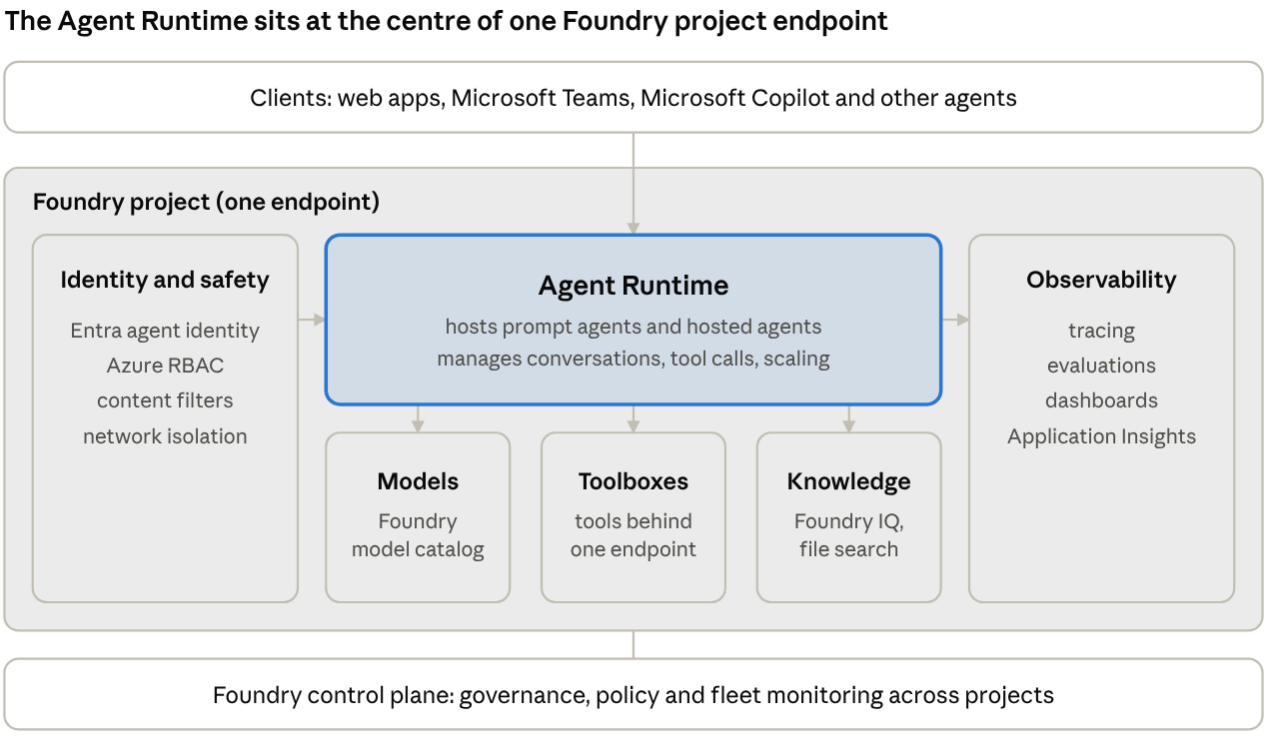


Figure 5 · Foundry Agent Service cloud architecture

Capability	How the service provides it
Agent creation and configuration	Versioned prompt-agent definitions, or hosted-agent container images
Model deployment and integration	Catalog models called through the Responses API on the project endpoint
Tool connections	Built-in tools, functions, OpenAPI and MCP; a Toolbox exposes tools through one MCP endpoint
Knowledge and data integration	File search, Azure AI Search and Foundry IQ permission-aware knowledge bases
Identity and authentication	A Microsoft Entra agent identity per agent, with access granted through RBAC
Managed execution	The Agent Runtime; hosted agents scale per session in VM-isolated sandboxes
Monitoring, tracing and evaluation	Traces in Application Insights, evaluations and dashboards
Security and governance	Content filters, guardrails, network isolation and the Foundry control plane

Memory, browser automation, computer use and the SharePoint and Fabric connectors are in preview (capability reference).

9. Key Features of Foundry Agent Service

Each feature removes a piece of platform work that teams would otherwise build themselves.

Feature	What the service provides, with an example
Managed hosting and execution	A managed endpoint and automatic scaling: a retail assistant absorbs a sale-day surge with no capacity planning
Azure and Microsoft integration	Publishing to Teams and Microsoft Copilot: an HR agent answers employees inside Teams
Tool calling and external APIs	Functions, OpenAPI tools and MCP servers: a logistics agent calls a carrier's REST API for shipment status
Enterprise authentication and security	Entra agent identity, RBAC and content filters: a finance agent may read one storage account and nothing else
Observability and monitoring	Traces of every model and tool call: an engineer sees why the wrong tool was called
Scalability and deployment	Immutable versions and CI/CD: a pipeline deploys a new version after tests pass
Knowledge integration	File search, Azure AI Search and Foundry IQ: a policy assistant returns only documents the user may see

Hosted agents can expose the Responses, Invocations, Activity (Teams and Microsoft 365) and A2A protocols. Foundry's visual workflow builder is in preview and retires on December 1, 2026; Microsoft recommends Agent Framework for new multi-agent workflows.

10. How Foundry Agent Service Works

An agent moves through seven steps, from an empty project to a monitored production endpoint.

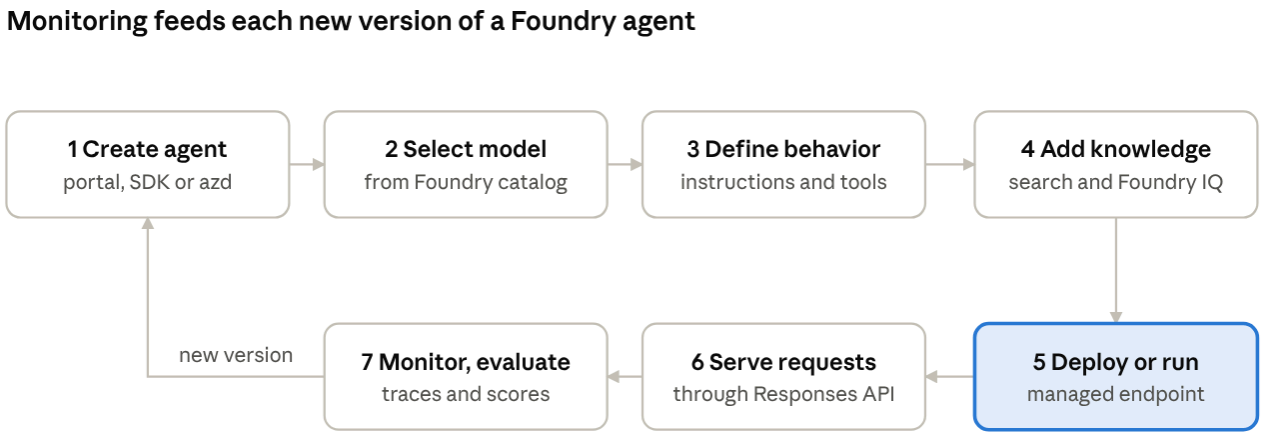


Figure 6 · Foundry Agent Service lifecycle: seven steps and a feedback loop

1. **Create an agent.** Define a prompt agent in the portal or with `project.agents.create_version(...)`, or start a hosted agent from a sample.
2. **Select and configure a model.** Deploy a model from the Foundry catalog and reference its deployment name.
3. **Define instructions and tools.** Write instructions and attach built-in tools, functions, OpenAPI specifications or MCP servers.
4. **Connect knowledge sources.** Add file search, an Azure AI Search index or a Foundry IQ knowledge base.
5. **Deploy or run the agent.** A prompt agent runs once saved. A hosted agent is deployed with `azd deploy`; Foundry builds it, assigns an identity and exposes an endpoint.
6. **Process user requests.** Applications call the agent through the Responses API; the runtime manages the conversation, tool calls and content filters.
7. **Monitor and evaluate performance.** Traces, evaluations and dashboards show quality, usage and cost.

The hosted agent quickstart uses an Agent Framework sample and currently relies on prerelease tooling.

11. MAF vs Microsoft Foundry Agent Service

Microsoft Agent Framework is a development and orchestration framework; Foundry Agent Service is a managed platform service. One is a library you write code with, the other runs agents in the cloud, so they complement each other.

Dimension	Microsoft Agent Framework	Foundry Agent Service
Purpose	Build agents and multi-agent workflows in code	Host, secure and operate agents

Dimension	Microsoft Agent Framework	Foundry Agent Service
Primary functionality	Agent abstraction, tool loop, middleware, workflows	Runtime, endpoints, identity, tracing, publishing
Development capabilities	Full code control in .NET, Python and Go	No-code prompt agents; hosted agents from any framework
Hosting and execution	None of its own; runs wherever your code runs	Fully managed on Azure
Agent orchestration	Sequential, concurrent, handoff, group chat, Magentic, custom graphs	Runs the orchestration your hosted code defines; A2A between agents
Model integration	Foundry, Azure OpenAI, OpenAI, Anthropic, Ollama and others	Foundry model catalog through the Responses API
Tool integration	Function tools, MCP, hosted tools, agents as tools	Built-in tools, OpenAPI, MCP, Toolbox
Security	Tool approval and middleware; you supply credentials	Entra agent identity, RBAC, content filters, network isolation
Monitoring	Emits OpenTelemetry to a backend you choose	Built-in tracing, evaluations and dashboards
Scalability	Set by your own infrastructure	Automatic; hosted agents scale per session
Typical use cases	Custom multi-agent systems; agents inside existing apps; multi-cloud	Governed production agents; agents published to Teams and Copilot

The two meet at the hosted agent: Agent Framework code deployed to Foundry gains an endpoint, identity, scaling and tracing. Neither requires the other. The framework runs outside Azure, and Foundry hosts agents written in other frameworks.

12. Integration and Real-World Architecture

Developers write the agent's logic with Agent Framework and deploy it to Foundry Agent Service as a hosted agent. The framework owns reasoning and orchestration; Foundry owns models, tools, identity, compute and operations.

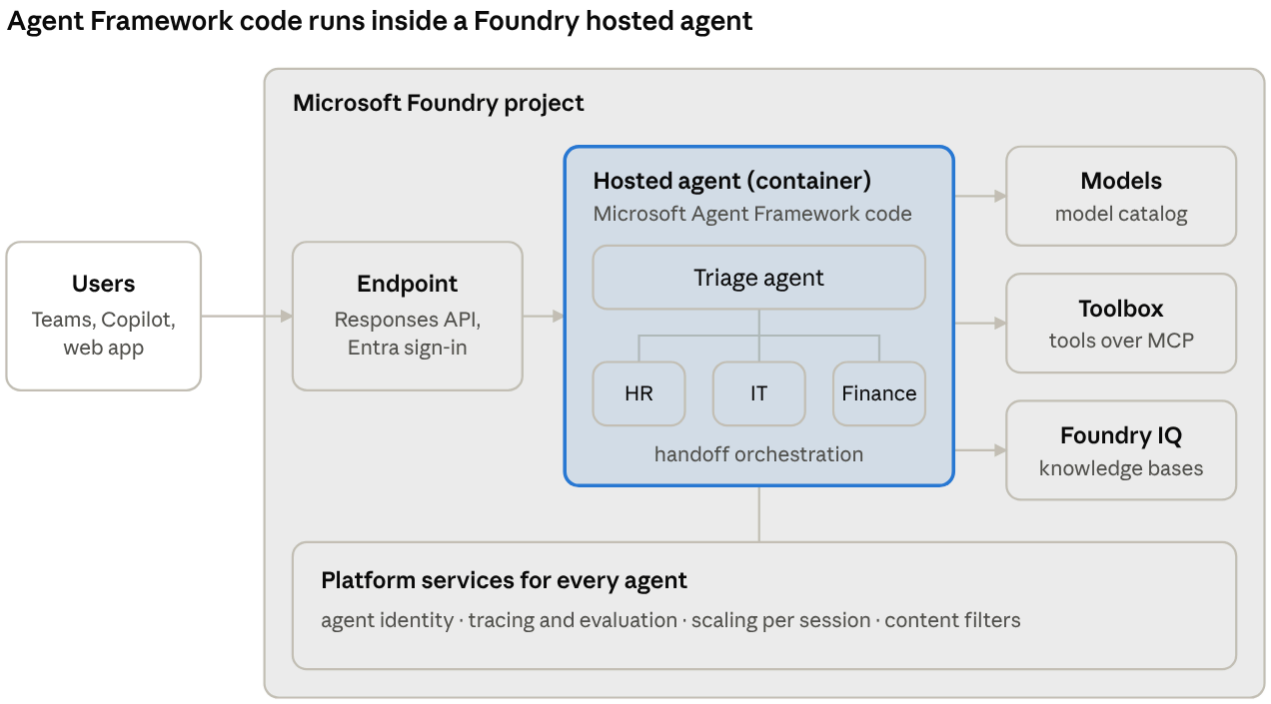


Figure 7 · integration architecture: Agent Framework agents hosted on Foundry Agent Service

- **Build.** Write agents and workflows in Python or C#, using `FoundryChatClient` so development and production share the same models.
- **Connect.** Reach shared tools through a Foundry Toolbox (`FoundryToolbox` in Python, `AddFoundryToolboxes` in .NET) and knowledge through Foundry IQ.
- **Deploy.** Test locally with `azd ai agent run`, then run `azd deploy`.
- **Operate.** Foundry adds the agent identity, per-session scaling, OpenTelemetry traces in Application Insights, immutable versions and content filters.

Practical example: an enterprise AI assistant

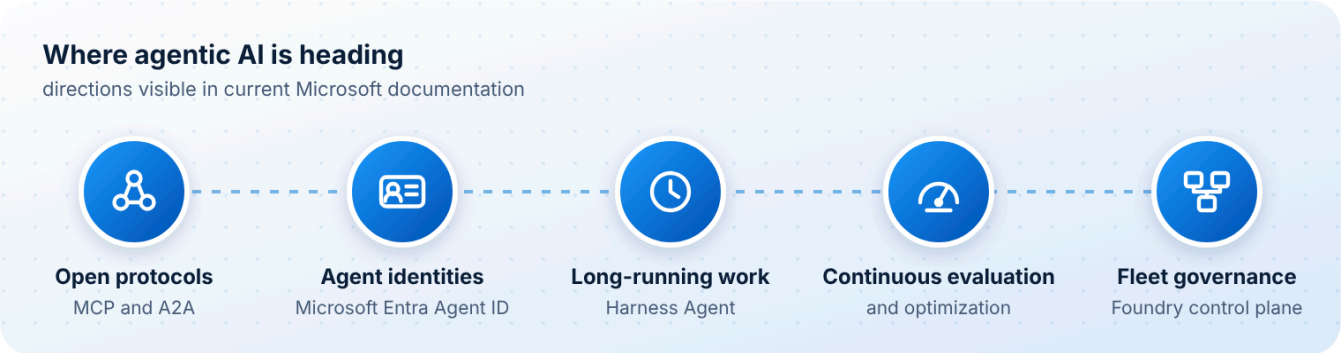
This illustrative design shows an employee assistant for a mid-sized company. It is a teaching example, not a Microsoft reference architecture.

The assistant is published to Microsoft Teams and runs as one hosted agent. A triage agent hands off to three specialists: HR answers policy questions from a Foundry IQ knowledge base over SharePoint, IT opens tickets through an OpenAPI tool, and Finance needs human approval for large reimbursements. When an employee asks about leave and a laptop repair, both actions appear in one trace.

13. Advantages, Limitations and Future Scope

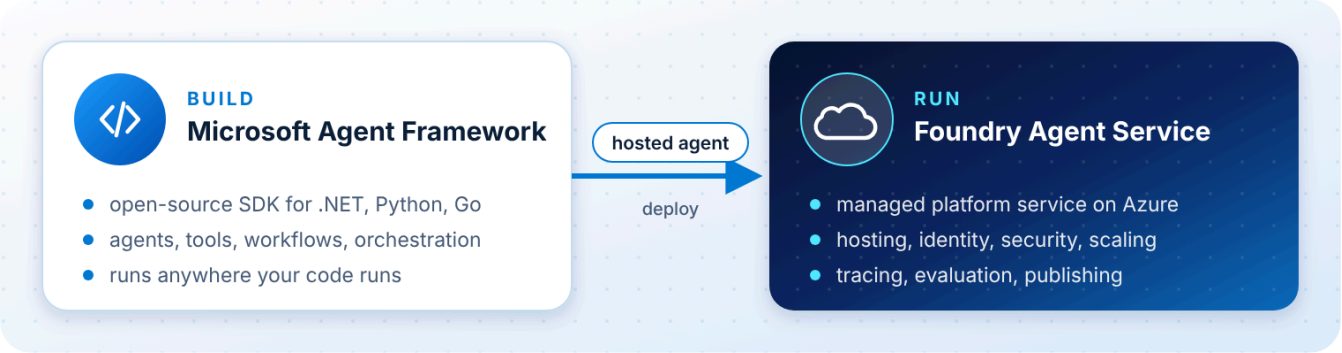
The ecosystem offers a complete path from prototype to governed production system, at the cost of fast-moving APIs and a broad surface to learn.

Topic	Assessment
Ecosystem benefits	One path: write with Agent Framework, run on Agent Service, publish to Teams and Copilot
Developer productivity	A few lines create an agent; orchestration, tool calling, sessions and telemetry are built in
Enterprise readiness	Identity, RBAC, network isolation, content filters, tracing and evaluation are platform features
Multi-agent possibilities	Five orchestration patterns, checkpointed workflows, agents as tools, A2A across services
Security considerations	Prompt injection through retrieved content; grant least privilege, require approval for consequential actions, keep secrets out of images
Learning curve	A first agent is quick; workflows, middleware, Azure identity and deployment take longer
Challenges and limitations	Fast-changing APIs; many preview features without an SLA; Go and hosted-agent language gaps; tool support varies by model provider; multi-agent runs multiply cost



14. Conclusion and References

Microsoft Agent Framework is how you write an agent; Foundry Agent Service is where you run and govern it. Together they cover the path from a first prototype to an enterprise system that can be secured, observed and scaled.



Key takeaways

- Choose an agent for open-ended tasks and a workflow for defined processes.
- Treat the framework and the service as layers of one solution, not as alternatives.
- Check the preview status of each feature before relying on it in production.
- Build tracing, evaluation and least-privilege access in from the start.